

Informe Tarea 1

Joaquín Rubilar García, jrubilar@usm.cl

Juan José Santibáñez Cuevas, jsantibanez@usm.cl

Sebastián Velásquez, svelasquezm@usm.cl

Tomás Abarca, tabarca@usm.cl

Iñaki Cenitagoya, icenitagoya@usm.cl

Diego Muñoz, dmunozz@usm.cl

Grupo 3

Resumen

La tarea se desarrolló en dos partes. En la primera se refactorizó una solución monolítica de sensores hacia una interfaz procedural basada en acciones, utilizando un enum para expresar con claridad las operaciones INIT, READ y SHOW. En la segunda parte se implementó una función de medición de tiempos de ejecución para comparar un ciclo vacío con ciclos que realizan operaciones de salida. La evidencia incluye diagramas de flujo, explicación de diseño, compilación mediante Makefile y diez mediciones independientes para cada caso.

1. Parte 1

1.1. Diseño de la función `lecturaSensor()` y de la interfaz `usarSensores()`

La función `lecturaSensor()` cumple el rol de módulo mínimo de adquisición simulada, no recibe parámetros y retorna una lectura aleatoria en el rango entero $[0, 255]$. En el código es `double`, aunque el dato operativo se interpreta como valor entero de milivoltios al almacenarse en el arreglo de sensores.

La mejora principal no está en aumentar artificialmente la complejidad de `lecturaSensor()`, sino en separar las fases del programa mediante el enum `Accion`. Se implementaron tres acciones cerradas, `INIT` para inicializar el arreglo en cero, `READ` para cargar una lectura simulada en cada posición y `SHOW` para mostrar los valores. Esta decisión evita repetir ciclos en `main()` y concentra el comportamiento de la interfaz de sensores en una sola función con `switch`, lo que mejora la legibilidad y reduce el acoplamiento del programa principal.

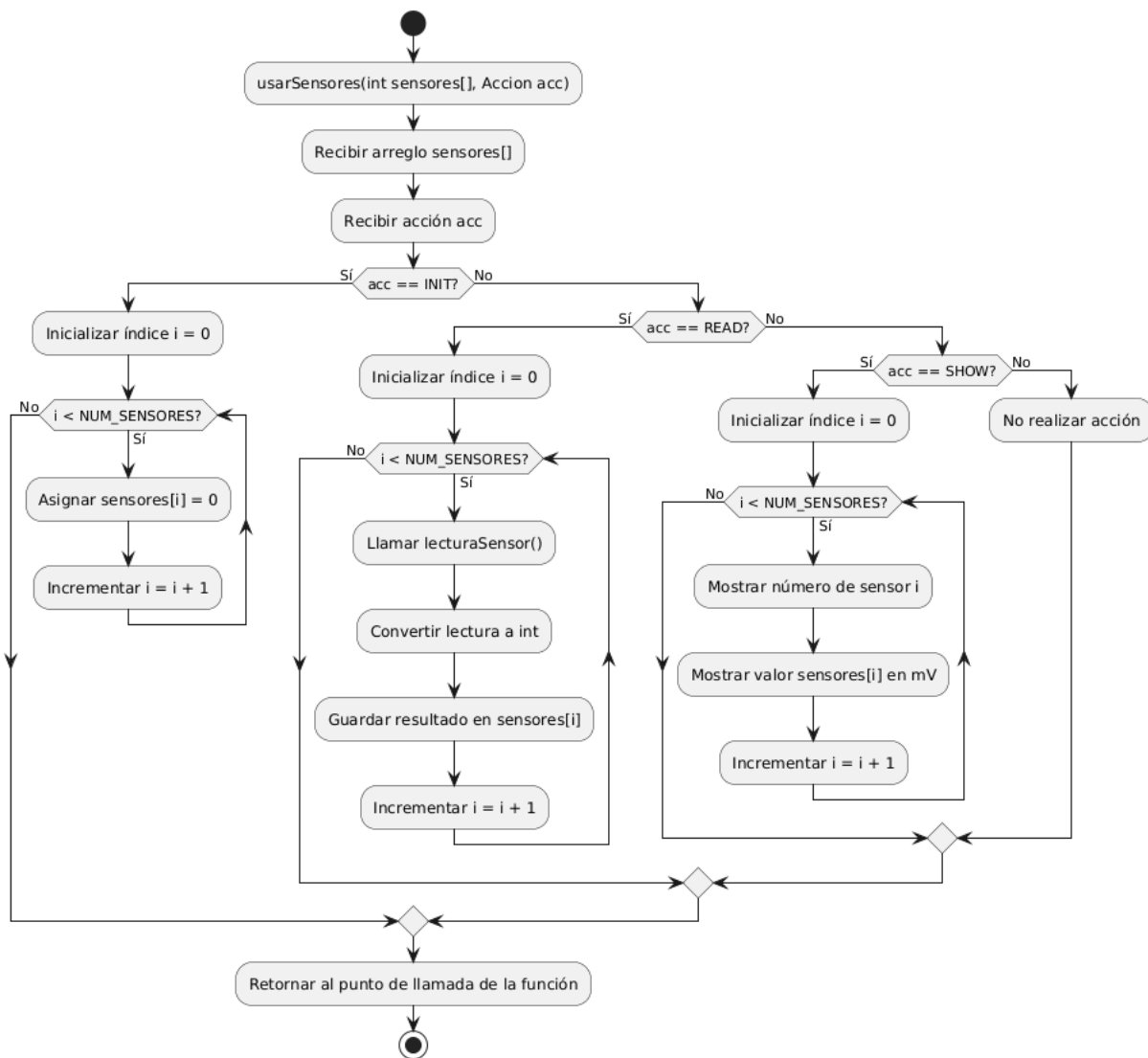


Figura 1. Diagrama de flujo de la función usarSensores().

La Figura 1 muestra que la función recibe el arreglo y la acción a ejecutar. Luego evalúa acc mediante switch. Si la acción es INIT, recorre el arreglo asignando cero. Si es READ, recorre el arreglo llamando a lecturaSensor() para cada sensor. Si es SHOW, recorre el arreglo mostrando el valor almacenado. El caso default se dejó como protección ante acciones no contempladas.

1.2. Diseño completo de la nueva función main()

La nueva función main() queda deliberadamente simple. Primero inicializa la semilla pseudoaleatoria con srand(time(NULL)), declara el arreglo sensores y luego delega el trabajo en la interfaz creada. La secuencia usarSensores(sensores, INIT), usarSensores(sensores, READ) y usarSensores(sensores, SHOW) mantiene el mismo comportamiento observable de la versión original, pero con una estructura más modular.

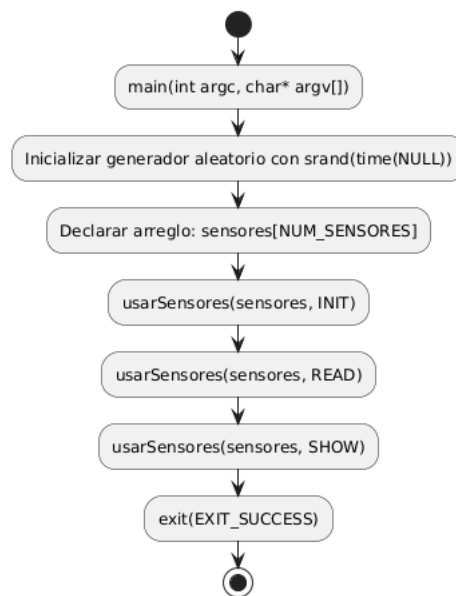


Figura 2. Diagrama de flujo de la nueva función `main()`.

Esta estructura es más mantenible que la versión monolítica porque `main()` ya no contiene tres ciclos separados. En términos de diseño procedural, `main()` queda como coordinador de alto nivel, mientras que `usarSensores()` implementa la lógica asociada a cada fase.

2. Parte 2

2.1. Hito 1: análisis de cicloEjemplo()

1) ¿Qué retorna `clock()`?

La función `clock()` retorna una aproximación del tiempo de la unidad de procesamiento central (CPU) consumido por el proceso desde un punto de referencia definido por la implementación. No debe interpretarse como tiempo cronológico absoluto tradicional, sino como una cantidad de ticks de reloj asociados al proceso.

2) ¿Qué unidades tienen inicio y fin?

En la implementación final se usó el tipo `clock_t` para inicio y fin. Sus valores están expresados en ticks de reloj de CPU. Por sí solos no están en segundos ni en nanosegundos.

3) ¿Por qué se divide por `CLOCKS_PER_SEC`?

Porque `CLOCKS_PER_SEC` indica cuántos ticks de `clock()` equivalen a un segundo. La diferencia `fin - inicio` entrega ticks; al dividir por `CLOCKS_PER_SEC` se obtiene el tiempo en segundos. Luego se multiplica por `1e9` para expresar el resultado en nanosegundos.

4) ¿Qué hace `(double)` como prefijo de `(fin - inicio)`?

Es una conversión explícita de tipo. Convierte el resultado de la resta de ticks a `double` antes de realizar la división.

5) ¿Para qué sirve usar (double) en esa expresión?

Sirve para forzar aritmética flotante, tal que permita decimales, evitando así la división entera. Sin ese cast, la parte decimal se podría truncar antes de convertir el resultado final, perdiendo precisión especialmente en mediciones pequeñas como éstas.

2.2. Hito 2: diseño de cicloPrueba(caso_t caso)

La función cicloPrueba(caso_t caso) se diseñó como una función de medición basada directamente en la estructura entregada en cicloEjemplo(). Para seleccionar el tipo de prueba se utilizó el enum caso_t, cuyos valores son VACIO, ESPACIO, ASTERISCO y NEWLINE. Según el valor recibido, la función ejecuta un ciclo for con el cuerpo correspondiente: una instrucción vacía, la impresión de un espacio, la impresión de un asterisco o la impresión de un salto de línea. En todos los casos, la medición comienza justo antes del ciclo y termina inmediatamente después, utilizando clock() para capturar los valores de inicio y fin.

Luego de ejecutar el ciclo, la diferencia entre fin e inicio se divide por CLOCKS_PER_SEC para convertir los ticks de reloj a segundos. Posteriormente, este valor se multiplica por 10^9 , obteniendo el tiempo total en nanosegundos. Finalmente, el tiempo total se divide por la cantidad de iteraciones, retornando así el tiempo promedio por iteración. De esta forma, la función mantiene el mismo criterio de medición para los cuatro casos y permite comparar el costo relativo de no realizar salida, imprimir un carácter simple e imprimir un salto de línea.

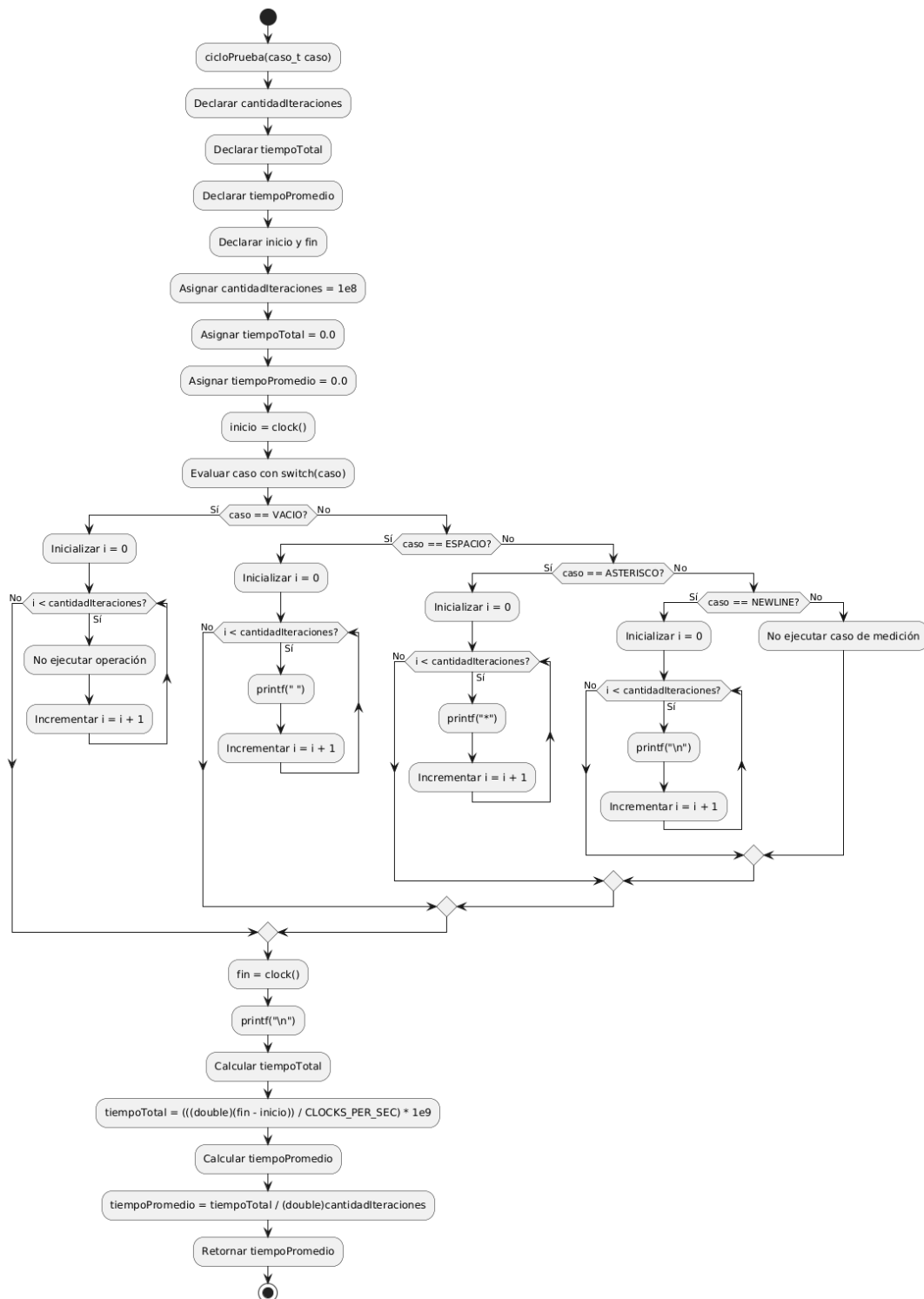


Figura 3. Diagrama de flujo de la función cicloPrueba(caso).

En la implementación final se utilizaron 100000000 iteraciones para cada uno de los casos medidos. Esta decisión permite mantener una comparación directa entre VACIO, ESPACIO, ASTERISCO y NEWLINE, ya que todos los tiempos promedio fueron obtenidos

bajo la misma cantidad de repeticiones. A partir de las diez ejecuciones realizadas en el Mac mini M4, se obtuvieron promedios aproximados de 0.76 [ns] para VACIO, 31.93 [ns] para ESPACIO, 32.18 [ns] para ASTERISCO y 329.05 [ns] para NEWLINE. Estos resultados muestran que las operaciones de salida por pantalla tienen un costo considerablemente mayor que un ciclo vacío, y que el salto de línea es el caso más costoso debido a su interacción más intensa con el sistema de salida estándar.

3.1 Hito 3: metodología experimental y resultados

El programa de la parte 2 fue compilado con el Makefile entregado, usando g++ con la bandera -std=c++17 -Wall. El entorno de prueba usado para las mediciones fue macOS Sequoia 15.7.4 de arquitectura ARM, con el Apple M4. Cada ejecución del programa entrega cuatro tiempos promedio, uno por cada caso. El programa se ejecutó diez veces de forma independiente y luego se calcularon el promedio aritmético y la desviación estándar muestral de cada caso.

Medición	VACIO [ns]	ESPACIO [ns]	ASTERISCO [ns]	NEWLINE [ns]
1	0.785830	31.918940	33.273480	337.212250
2	0.761170	32.670170	33.028380	336.900640
3	0.766710	31.944680	31.920540	339.845640
4	0.619330	31.884920	31.986290	330.989620
5	0.760170	31.784070	31.904910	332.153700
6	0.776590	32.026500	31.952790	329.814780
7	0.778100	31.592150	31.851890	327.121510
8	0.763500	31.741970	31.913530	319.170740
9	0.757240	31.913860	32.113090	319.266550
10	0.781460	31.772860	31.859950	318.059820
Promedio	0.755010	31.925012	32.180485	329.053525
Desv. estándar	0.048689	0.289642	0.519953	8.004161

Tabla 1. Diez mediciones independientes y estadísticos agregados por caso.

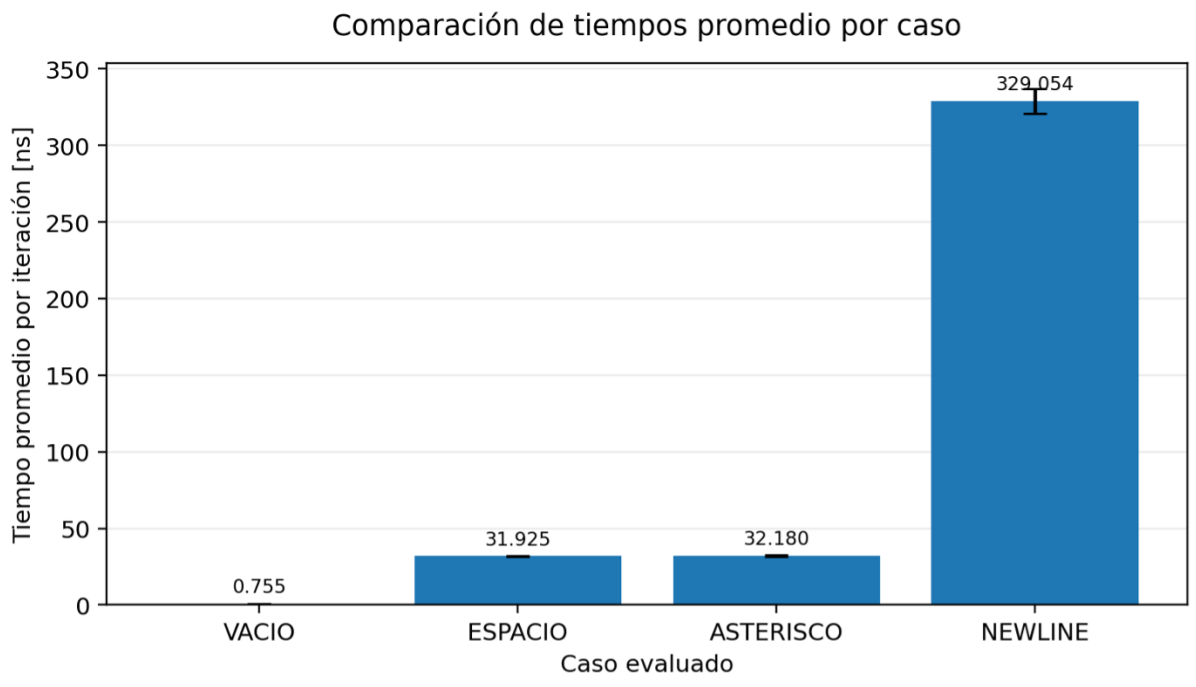


Figura 4. Comparación de tiempos promedio en escala logarítmica.

Respecto a las tendencias observadas a partir de la Tabla 1 y Figura 4, se deduce que el caso VACIO obtuvo el menor tiempo promedio (0.755 [ns]), lo que es coherente porque el ciclo no realiza operaciones de salida. Los casos ESPACIO y ASTERISCO fueron muy similares (31.925 [ns] y 32.180 [ns], respectivamente), lo que indica que el costo dominante no depende del carácter específico impreso, sino de la llamada a la función de salida y del manejo de stdout. El caso NEWLINE fue claramente el mayor (329.054 [ns]), aproximadamente 10.3 veces más lento que ESPACIO y 10.2 veces más lento que ASTERISCO.

En el mismo sentido, se puede interpretar de los resultados evidencian que una operación que solo recorre un ciclo tiene un costo muy inferior al de una operación que interactúa con el sistema de entrada/salida. En particular, imprimir un espacio o un asterisco implica una ruta más costosa que una iteración vacía, porque intervienen printf, el buffer de salida, la biblioteca estándar y el sistema operativo. El salto de línea aumenta aún más el tiempo promedio, probablemente porque modifica el comportamiento del flujo de salida y puede generar trabajo adicional asociado al manejo de líneas en la terminal.

La diferencia relevante no está en cada medición aislada, sino en la separación entre órdenes de magnitud. Con los nuevos datos, el caso NEWLINE obtuvo un tiempo promedio de aproximadamente 329.05 [ns], mientras que ESPACIO fue de 31.93 [ns] y VACIO de 0.76 [ns]. Esto significa que NEWLINE fue aproximadamente 10.3 veces más lento que ESPACIO y aproximadamente 436 veces más lento que VACIO. Además, ESPACIO y ASTERISCO presentaron tiempos casi equivalentes, lo que es coherente

porque ambos casos imprimen un único carácter por iteración. La mayor desviación estándar de NEWLINE también es esperable, ya que el salto de línea introduce una interacción más costosa y variable con la salida estándar, el sistema operativo y la terminal, a diferencia del ciclo vacío, cuyo comportamiento depende principalmente de operaciones locales del procesador.

3. 4. Conclusión

A partir de los experimentos realizados con la función cicloPrueba, se concluye que el rendimiento de las operaciones de salida en C no depende únicamente de la cantidad de datos, sino fundamentalmente de la gestión de memoria intermedia (buffering) y la frecuencia de las llamadas al sistema. Los resultados demuestran que caracteres simples como el espacio o el asterisco presentan un costo temporal reducido debido a que el estándar de C los agrupa en un búfer antes de su procesamiento. En contraste, el uso de saltos de línea bajo una configuración de line buffering incrementa drásticamente la latencia.

Finalmente, el uso de cargas de trabajo diferenciadas y la normalización de tiempos en nanosegundos permitieron validar el experimento frente a las limitaciones de resolución del reloj del sistema. Este laboratorio subraya la importancia de comprender la arquitectura subyacente y el funcionamiento de las bibliotecas estándar, concluyendo que una correcta configuración del flujo de datos es tan vital para la eficiencia de un sistema como la optimización del algoritmo principal.

4. 5. Anexo A: fuente de los diagramas de flujo

A continuación se deja el código fuente equivalente usado para documentar los diagramas de flujo renderizados con código PlantUML. La lógica corresponde directamente a las funciones implementadas.

4.1. 5.1. usarSensores()

```
@startuml
start
:usarSensores(int sensores[], Accion acc);
:Recibir arreglo sensores[];
:Recibir acción acc;

if (acc == INIT?) then (Sí)
  :Inicializar índice i = 0;
  while (i < NUM_SENORES?) is (Sí)
    :Asignar sensores[i] = 0;
    :Incrementar i = i + 1;
  endwhile (No)
else (No)
  if (acc == READ?) then (Sí)
    :Inicializar índice i = 0;
    while (i < NUM_SENORES?) is (Sí)
      :Llamar lecturaSensor();
    endwhile (No)
  else (No)
  endif (No)
endif (No)
```



```
        :Convertir lectura a int;
        :Guardar resultado en sensores[i];
        :Incrementar i = i + 1;
    endwhile (No)

    else (No)
        if (acc == SHOW?) then (Sí)
            :Inicializar índice i = 0;
            while (i < NUM_SENTORES?) is (Sí)
                :Mostrar número de sensor i;
                :Mostrar valor sensores[i] en mV;
                :Incrementar i = i + 1;
            endwhile (No)

            else (No)
                :No realizar acción;
            endif
        endif
    endif

    :Retornar al punto de llamada de la función;
    stop

@enduml
```

4.2. 5.2. main()

```
@startuml

start
:main(int argc, char* argv[]);
:Inicializar generador aleatorio con srand(time(NULL));
:Declarar arreglo: sensores[NUM_SENTORES];

:usarSensores(sensores, INIT);
:usarSensores(sensores, READ);
:usarSensores(sensores, SHOW);

:exit(EXIT_SUCCESS);
stop

@enduml
```

4.3. 5.3. cicloPrueba(caso)

```
@startuml

start
:cicloPrueba(caso_t caso);

:Declarar cantidadIteraciones;
:Declarar tiempoTotal;
:Declarar tiempoPromedio;
:Declarar inicio y fin;

:Asignar cantidadIteraciones = 1e8;
:Asignar tiempoTotal = 0.0;
:Asignar tiempoPromedio = 0.0;

:inicio = clock();

:Evaluar caso con switch(caso);

if (caso == VACIO?) then (Sí)
```

```
:Inicializar i = 0;
while (i < cantidadIteraciones?) is (Sí)
    :No ejecutar operación;
    :Incrementar i = i + 1;
endwhile (No)

else (No)
    if (caso == ESPACIO?) then (Sí)
        :Inicializar i = 0;
        while (i < cantidadIteraciones?) is (Sí)
            :printf(" ");
            :Incrementar i = i + 1;
        endwhile (No)

    else (No)
        if (caso == ASTERISCO?) then (Sí)
            :Inicializar i = 0;
            while (i < cantidadIteraciones?) is (Sí)
                :printf("*");
                :Incrementar i = i + 1;
            endwhile (No)

        else (No)
            if (caso == NEWLINE?) then (Sí)
                :Inicializar i = 0;
                while (i < cantidadIteraciones?) is (Sí)
                    :printf("\n");
                    :Incrementar i = i + 1;
                endwhile (No)

            else (No)
                :No ejecutar caso de medición;
            endif
        endif
    endif
endif

:fin = clock();
:printf("\n\n");

:Calcular tiempoTotal;
:tiempoTotal = (((double)(fin - inicio)) / CLOCKS_PER_SEC) * 1e9;

:Calcular tiempoPromedio;
:tiempoPromedio = tiempoTotal / (double)cantidadIteraciones;

:Retornar tiempoPromedio;
stop

@enduml
```